Laporan Tugas Kecil 2

IF2211 Strategi Algoritma

Voxelization Objek 3D menggunakan Octree

Semester II Tahun 2025/2026

Oleh:

An-Dafa Anza Avansyah (13524038)

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

28 Maret 2026

Daftar Isi

1 Pendahuluan **4**

2 Dasar Teori **5**
2.1 *Voxelization* 5
2.2 Octree 5
2.3 Algoritma Divide and Conquer 5
2.4 *Separating Axis Theorem* 6
2.5 Konkurensi dan *Multithreading* 6

3 Desain dan Implementasi **8**
3.1 Algoritma Divide and Conquer pada Voxelization 8
3.2 Implementasi Konkurensi (*Multithreading*) 9
3.3 Struktur Program 10
3.4 *Source Code* 10
3.4.1 Octree 11
3.4.2 geometry.hpp 16
3.4.3 main.cpp 17
3.4.4 viewer.cpp 20

4 Eksperimen **25**
4.1 Testcases 25
4.2 Hasil 27

5 Penutup **33**
5.1 Kesimpulan 33
5.2 Surat Cinta dari Waifu 33

Lampiran **35**

Daftar Gambar

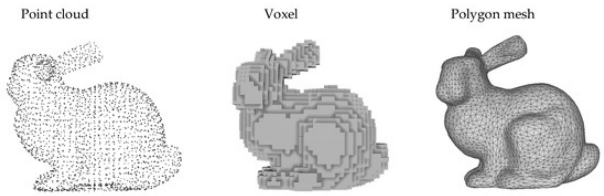
1 *Point cloud, voxel, and polygon mesh representation of 3D models* 4

Listings

1 Fungsi BuildRecursive 8
2 octree.h 11
3 octree.cpp 12
4 geomtery.hpp 16
5 main.cpp 18
6 viewer.cpp 20

1 Pendahuluan

Voxelization merupakan proses konversi objek tiga dimensi yang umum menjadi sebuah model yang tersusun dari kubus-kubus kecil seperti di Minecraft. Proses konversi ini dilakukan dengan memanfaatkan struktur data Octree, yaitu sebuah tree yang setiap percabangan terdiri dari 8 anak. Melalui Octree, sebuah ruang kubus 3D dapat dipotong menjadi 8 kubus berbeda, yang masing-masing dapat dipotong lagi menjadi 8 kubus yang lebih kecil.



Gambar 1: *Point cloud, voxel, and polygon mesh representation of 3D models*

Dalam pemodelan voxel-based, sebuah voxel merepresentasikan unit terkecil dari volume benda padat (bisa bernilai biner 0 dan 1 yang menandakan ruang tersebut kosong atau terisi). Penggunaan struktur data voxel ini memberikan keuntungan yang signifikan dalam komputasi spasial, karena bentuknya yang seragam memudahkan operasi boolean (*Solid Geometry*), pendeteksian benturan (*collision detection*), serta perhitungan volume dan massa. Oleh karena itu, teknik vokselisasi sangat krusial dan banyak diaplikasikan pada berbagai bidang, mulai dari pencitraan medis (seperti data hasil pemindaian MRI atau CT Scan), manufaktur pencetakan 3D, hingga simulasi fluida komputasional (*Computational Fluid Dynamics*).

Pada tugas ini kecil 2 IF2211 Strategi Algoritma, penulis merancang dan mengimplementasikan sebuah program 3D Voxelizer menggunakan bahasa pemrograman C++. Program ini dirancang berjalan melalui Command Line Interface (CLI) untuk membaca model 3D geometris kontinu dalam format file .obj (yang terdiri dari vertices dan faces berupa poligon segitiga), lalu mengonversinya menjadi representasi spasial diskrit berupa kubus-kubus kecil berukuran seragam (voxel).

2 Dasar Teori

2.1 *Voxelization*

Vokselisasi adalah teknik fundamental dalam grafika komputer dan pemodelan geometris yang berfungsi untuk mendiskretisasi ruang kontinu tiga dimensi. Istilah voxel (singkatan dari *volumetric pixel*) merepresentasikan sebuah titik nilai atau elemen volume pada ruang grid 3D yang beraturan. Berbeda dengan piksel 2D yang merepresentasikan kotak pada bidang datar, voxel berbentuk kubus yang memiliki dimensi panjang, lebar, dan tinggi.

Dalam praktiknya, vokselisasi dari sebuah model poligon 3D (seperti objek berekstensi `.obj`) memetakan permukaan (*surface*) atau volume internal benda tersebut ke dalam struktur data grid. Terdapat dua pendekatan utama dalam vokselisasi:

1. Surface Voxilization: Hanya mengonversi batas atau cangkang permukaan objek menjadi deretan voxel. Pendekatan ini umum digunakan untuk rendering visual.

2. Solid Voxelization: Mengisi seluruh bagian dalam (volume internal) dari objek geometris dengan voxel.

2.2 Octree

Octree adalah struktur data berbasis pohon (*tree*) hierarkis yang digunakan untuk mempartisi ruang tiga dimensi secara spasial. Konsep dasar Octree adalah perluasan dari *Quadtree* (yang digunakan pada ruang 2D). Pada struktur Octree, setiap node cabang (internal node) akan selalu membelah ruang (biasanya berupa Bounding Box berbentuk kubus) menjadi tepat delapan bagian (octants) yang sama besar.

Proses pembelahan ini dilakukan secara rekursif hingga mencapai kriteria penghentian tertentu, seperti tercapainya batas kedalaman pohon (max depth), batas resolusi minimum kubus, atau kondisi di mana node sudah sepenuhnya kosong atau sepenuhnya terisi.

Keunggulan utama Octree dalam proses vokselisasi terletak pada kemampuannya untuk mengompresi memori dan mempercepat komputasi ruang kosong (*sparse data*). Melalui teknik pemangkasan (*pruning*), jika sebuah kubus partisi (*node*) diketahui tidak bersinggungan (*intersect*) dengan poligon model objek sama sekali, maka program tidak perlu menelusuri atau membelah kubus tersebut lebih jauh ke bawah.

2.3 Algoritma Divide and Conquer

Pada mulanya algoritma *Divide and Conquer* adalah strategi militer yang dikenal dengan *divide ut imperes* atau bahasa yang lebih dikenal oleh kalangan umum adalah adu domba. Alih-alih menyelesaikan persoalan secara utuh, algoritma *Divide and Conquer* bekerja dengan membagi persoalan menjadi sub-persoalan yang lebih kecil, umumnya dibagi menjadi dua sub-persoalan yang ukurannya hampir sama dan menyelesaikan masing-masing sub-persoalan tersebut.

Terdapat tiga komponen dalam algoritma ini. Komponen pertama adalah *divide*, yakni membagi persoalan menjadi beberapa sub-persoalan yang memiliki kemiripan denga persoalan semula namun berukuran lebih kecil (idealnya setiap sub-persolan berukuran

hampir sama). Komponen kedua adalah *conquer*, yakni menyelesaikan masing-masing sub-persoalan (secara langsung jika sudah berukuran kecil atau secara rekursif jika masih berukuran besar). Dan komponen yang ketiga adalah *combine*, yakni menggabungkan solusi masing-masing upa-persoalan sehingga membentuk solusi persoalan semula.

Dalam konteks program vokselisasi yang dibangun, Divide and Conquer diimplementasikan secara langsung melalui struktur Octree. Tahap *Divide* terjadi saat ruang Bounding Box raksasa dibelah menjadi 8 kubus anak. Tahap *Conquer* adalah saat setiap kubus anak diuji secara independen (bahkan secara paralel menggunakan *multithreading*) untuk melihat apakah mereka menabrak segitiga model 3D. Terakhir, tahap *Combine* terjadi secara tersirat saat seluruh kubus berukuran terkecil (*leaf nodes*) yang valid disatukan ke dalam sebuah koleksi data akhir sebagai representasi voxel keseluruhan objek.

2.4 *Separating Axis Theorem*

Separating Axis Theorem (SAT) adalah salah satu algoritma matematis fundamental dalam grafika komputer dan pendeteksian benturan (collision detection). Teorema ini menyatakan bahwa jika dua objek cembung (convex) tidak saling bersinggungan di ruang 3D, maka pasti terdapat setidaknya satu sumbu (sebuah garis lurus) di mana proyeksi ortogonal dari kedua objek tersebut tidak saling tumpang tindih (overlap). Sumbu ini disebut sebagai "Sumbu Pemisah" *(Separating Axis*). Sebaliknya, jika proyeksi kedua objek saling tumpang tindih di semua sumbu yang mungkin diuji, maka kedua objek tersebut pasti saling bersinggungan.

Dalam konteks vokselisasi, SAT digunakan untuk mendeteksi irisan (intersection) antara Axis-Aligned Bounding Box (AABB), yang merepresentasikan node berbentuk kubus pada Octree, dengan poligon segitiga dari model 3D. Menguji setiap titik di dalam ruang kubus adalah hal yang mustahil secara komputasional. Oleh karena itu, SAT menyederhanakan masalah ini dengan hanya menguji maksimal 13 sumbu utama, yang terdiri dari:3 sumbu normal dari kubus, 1 sumbu normal dari bidang segitiga, 9 sumbu yang dihasilkan dari Cross Product antara 3 sisi batas kubus dan 3 sisi segitiga.

2.5 Konkurensi dan *Multithreading*

Konkurensi adalah kemampuan suatu sistem komputer untuk mengeksekusi beberapa tugas atau proses secara bersamaan yang saling tumpang tindih dalam satu waktu. Dalam konteks pemrograman modern, konkurensi sering kali diimplementasikan menggunakan multithreading, di mana sebuah proses utama dipecah menjadi beberapa thread eksekusi yang lebih kecil yang dapat berjalan secara paralel pada prosesor *multi-core*.

Pemrosesan paralel sangat ideal diterapkan pada algoritma yang berfokus pada *Data Parallelism*, yaitu ketika operasi yang sama harus dilakukan pada kumpulan data yang berbeda secara independen. Dalam kasus struktur Octree, setiap node cabang dapat diproses dan dibelah tanpa harus menunggu node cabang lainnya selesai.

Namun, implementasi multithreading memunculkan tantangan berupa Race Condition, yaitu kondisi di mana dua atau lebih thread mencoba membaca dan memodifikasi memori yang sama secara bersamaan, sehingga menghasilkan data yang korup atau tidak valid. Untuk mencegah hal ini, digunakan mekanisme sinkronisasi yang disebut *Mutual Exclusion* (Mutex). Mutex berfungsi sebagai kunci yang memastikan bahwa hanya

ada satu thread yang dapat mengakses atau mengubah *Shared Memory* (seperti variabel penghitung/counter atau vektor penyimpan voxel) pada satu waktu tertentu, sementara thread lainnya harus mengantre.

3 Desain dan Implementasi

Bab ini membahas perancangan dan implementasi program, yang mencakup algoritma, struktur direktori, *source code*, serta penjelasan kelas-kelas penting yang digunakan dalam program.

3.1 Algoritma Divide and Conquer pada Voxelization

Algoritma Divide and Conquer menjadi inti dari proses vokselisasi dalam program ini. Alih-alih mengecek setiap kemungkinan koordinat 3D satu per satu (yang akan sangat lambat dan memakan memori), program memulai dari satu ruang pembatas (*Bounding Box*) raksasa yang membungkus keseluruhan model 3D, lalu memecahnya secara rekursif.

Langkah-langkah algoritma *Divide and Conquer* dalam program ini dapat dijabarkan sebagai berikut:

1. *Divide*: Jika sebuah ruang kubus (*node*) terdeteksi bersinggungan dengan model 3D dan belum mencapai batas kedalaman maksimum (*max depth*), ruang tersebut dibelah menjadi 8 kubus anak (*occtans*) yang ukurannya sama besar.

2. *Conquer*: Setiap kubus anak kemudian dievaluasi secara independen. Program mengecek apakah kubus tersebut bersinggungan dengan poligon segitiga pembentuk model 3D menggunakan metode *Separating Axis Theorem* (SAT). Jika tidak ada singgungan sama sekali, program melakukan pemangkasan dan berhenti menelusuri cabang tersebut.

3. *Combine*: Jika rekursi telah mencapai keadaan kedalaman maksimum dan kubus tersebut valid (bersinggungan dengan model), kubus tersebut disimpan ke dalam struktur data vektor bersama. Pada akhirnya, seluruh kubus kecil ini digabungkan menjadi representasi utuh dari objek (kumpulan voxel).

Berikut adalah *pseudocode* dari algoritma *Divide and Conquer* yang diimplementasikan pada fungsi rekursif pembangun Octree:

```
Function BuildRecursive(Box ruang, List<Segitiga> model, Integer
   kedalaman):
    // 1. Catat bahwa node baru terbentuk
    hitungNodeTerbentuk(kedalaman) += 1

    // 2. Kumpulkan segitiga yang bersinggungan dengan ruang ini
    List<Segitiga> irisanSegitiga = Kosong
    Untuk setiap Segitiga t dalam model:
        Jika cekIrisan(ruang, t) adalah True:
            Tambahkan t ke irisanSegitiga

    // 3. Jika kosong, lakukan pruning (berhenti)
    Jika irisanSegitiga NULL:
        hitungNodeDipangkas(kedalaman) += 1
    Return

```

```
    // 4. Jika sudah mencapai batas kedalaman, simpan sebagai Voxel (Combine)
    Jika kedalaman_saat_ini == BATAS_KEDALAMAN:
        Simpan ruang ke dalam List_Voxel_Global
    Return

    // 5. Divide: Belah ruang menjadi 8 bagian
    List<Box> anak = Belah_Menjadi_Delapan(ruang)

    // 6. Conquer: Lakukan rekursi untuk setiap anak
    Untuk setiap Box b dalam anak:
        BuildRecursive(b, irisanSegitiga, kedalaman + 1)
```

Listing 1: Fungsi BuildRecursive

3.2 Implementasi Konkurensi (*Multithreading*)

Proses vokselisasi pada model 3D dengan resolusi tinggi (kedalaman Octree yang besar) membutuhkan komputasi matematis yang sangat masif. Untuk mengoptimalkan waktu eksekusi program, algoritma ini dirancang untuk berjalan secara paralel menggunakan pustaka standar C++ `<future>` dan `<mutex>`.

Implementasi konkurensi pada program ini berfokus pada pendekatan Data *Parallelism* di tingkat pembelahan pertama pohon Octree, dengan mekanisme sebagai berikut:

1. Thread Asinkron (`std::async`)

   Alih-alih memproses seluruh node secara sekuensial, program memanfaatkan fakta bahwa pada level kedalaman ke-1, Bounding Box akar selalu terbelah menjadi tepat 8 node anak. Karena pengecekan irisan pada masing-masing kubus anak ini bersifat independen (tidak saling bergantung satu sama lain), program meluncurkan 8 tugas secara bersamaan menggunakan fungsi `std::async` dengan *policy* `std::launch::async`. Hal ini memaksa sistem operasi untuk mendelegasikan proses penelusuran (*buildRecursive*) tiap-tiap cabang Octree ke thread yang berbeda secara simultan.

2. Sinkronisasi dengan `std::future`

   Objek hasil dari `std::async` disimpan ke dalam wadah `std::vector<std::future<void»`. Hal ini berfungsi sebagai mekanisme sinkronisasi (*barrier*). Program utama akan memanggil fungsi `.get()` pada setiap objek future di dalam sebuah loop. Langkah ini memastikan bahwa fungsi `voxelize()` tidak akan selesai dan mengembalikan hasil sebelum seluruh kedelapan thread pembelahan Octree benar-benar tuntas mengeksekusi tugasnya hingga ke level leaf node terdalam.

3. Thread Safety dengan `std::mutex`

   Tantangan terbesar dalam *multithreading* adalah masalah tumpang tindih memori (*Race Condition*), yang terjadi ketika beberapa thread mencoba menulis data ke variabel yang sama di waktu yang bersamaan. Pada program ini, seluruh thread berbagi memori pada tiga variabel utama: wadah penyimpan hasil

(`std::vector<BoundingBox> voxels`) serta wadah pencatat statistik (nodesFormedCount dan nodesSkippedCount). Untuk mengatasi Race Condition tanpa mengubah struktur header file, program mengimplementasikan konsep *Mutual Exclusion* menggunakan `std::mutex` (bernama voxelMutex). Setiap kali sebuah thread menemukan node yang harus dipangkas, node yang terbentuk, atau leaf node yang valid untuk dimasukkan ke dalam daftar voxel, thread tersebut akan membungkus operasinya dengan `std::lock_guard<std::mutex>`. Lock guard ini akan "mengunci" akses variabel agar thread lain mengantre sejenak, sehingga integritas dan akurasi data tetap terjamin 100% meskipun diproses secara paralel.

3.3 Struktur Program

Program yang penulis buat ditulis dengan bahasa pemrograman C++ dengan menggunakan raylib sebagai sistem *Graphical User Interface* (GUI). Program ini dibangun dengan paradigma pemrograman orientasi objek yang diterapkan pada file `octree.h` serta paradigma pemrograman prosedural. Berikut adalah struktur *tree* program yang penulis buat:

```
`-- Tucil2_13524038/
+-- Makefile
+-- LICENSE
+-- README.md
+-- doc/
|   `-- Tucil2_13524038.pdf
+-- test/
|   `-- file.obj
+-- src/
|   +-- geometry.hpp
|   +-- main.cpp
|   +-- obj_parser.cpp
|   +-- obj_parser.hpp
|   +-- octree.cpp
|   +-- octree.hpp
|   `-- viewer.cpp
+-- lib/
|   `-- libraylib.a
+-- include/
|   +-- raylib.h
|   `-- raymath.h
`-- bin/
```

3.4 *Source Code*

Pada bab ini penulis akan menampilkan dua kode sumber penting yang menjadi dasar (*backbone*) dari program yang penulis buat, selain itu penulis juga akan menampilkan kode sumber `main.cpp` yang menjadi titik masuk program voxelization dijalankan.

3.4.1 Octree

Kelas ini merepresentasikan struktur data Octree yang diguankan. Kelas ini memiliki beberapa atribut *private* diantaranya adalah `maxDepth` yang merepresentasikan kedalaman maksimum octree, `voxels` yang merepresentasikan sebuah vector *bounding box*, voxelMutex yang digunakna untuk implementasi *mutual exclusion*, `nodesFormedCount` yang merepresentasikan vector untuk mendaftarkan node yang dikunjungi, serta `nodesSkippedCount` yang merepresentasikan vector untuk mendaftarkan node yang dilewati. Selain atribut, terdapat juga method private yakni `buildRecursive()` yang menjalankan algoritma *Divide and Conqueror* pada pembentukan octree, serta method `checkIntersection` yang berfungsi sebagai helper untuk pengecekan apabila node bersinggungan dengan model poligon.

Selain itu, kelas ini juga memiliki beberapa *public method*, seperti konstruktor, method `voxelize` untuk menjalankan algoritma utama voxelization, serta method getter seperti `getNodesFormed()` dan `getNodesSkipped()`.

1. octree.h

```cpp
#ifndef OCTREE_HPP
#define OCTREE_HPP

#include "geometry.hpp"
#include <vector>
#include <mutex>

class Octree {
    public:
    Octree(int maxDepth);
    void voxelize(const BoundingBox& rootBox, const std::vector<
        Triangle>& triangles);
    const std::vector<BoundingBox>& getVoxels() const;
    std::vector<int> getNodesFormed() const {
        return nodesFormedCount;
    }
    std::vector<int> getNodesSkipped() const {
        return nodesSkippedCount;
    }

    private:
    int maxDepth;
    std::vector<BoundingBox> voxels;
    std::mutex voxelMutex;
    std::vector<int> nodesFormedCount;
    std::vector<int> nodesSkippedCount;

    void buildRecursive(const BoundingBox& box, const std::
        vector<Triangle>& triangles, int currentDepth);
    std::vector<BoundingBox> divideBox(const BoundingBox& box);
    bool checkIntersection(const BoundingBox& box, const
        Triangle& tri);
```

```
};

#endif
```

Listing 2: octree.h

2. octree.cpp

```
#include "octree.hpp"
#include <cmath>
#include <future>
using namespace std;

Octree::Octree(int maxDepth) : maxDepth(maxDepth) {
    nodesFormedCount.assign(maxDepth + 1, 0);
    nodesSkippedCount.assign(maxDepth + 1, 0);
}

const vector<BoundingBox>& Octree::getVoxels() const {
    return voxels;
}

vector<BoundingBox> Octree::divideBox(const BoundingBox& box) {
    vector<BoundingBox> children(8);
    Vector3 center = box.getCenter();
    Vector3 min = box.min;
    Vector3 max = box.max;

    children[0].min = Vector3(min.x, min.y, min.z);
    children[0].max = Vector3(center.x, center.y, center.z);

    children[1].min = Vector3(center.x, min.y, min.z);
    children[1].max = Vector3(max.x, center.y, center.z);

    children[2].min = Vector3(min.x, center.y, min.z);
    children[2].max = Vector3(center.x, max.y, center.z);

    children[3].min = Vector3(center.x, center.y, min.z);
    children[3].max = Vector3(max.x, max.y, center.z);

    children[4].min = Vector3(min.x, min.y, center.z);
    children[4].max = Vector3(center.x, center.y, max.z);

    children[5].min = Vector3(center.x, min.y, center.z);
    children[5].max = Vector3(max.x, center.y, max.z);

    children[6].min = Vector3(min.x, center.y, center.z);
    children[6].max = Vector3(center.x, max.y, max.z);

    children[7].min = Vector3(center.x, center.y, center.z);
```

```
    children[7].max = Vector3(max.x, max.y, max.z);

    return children;
}

bool Octree::checkIntersection(const BoundingBox& box, const
   Triangle& tri) {
    Vector3 boxCenter = box.getCenter();
    Vector3 boxExtents = (box.max - box.min) * 0.5f;

    Vector3 v0 = tri.v0 - boxCenter;
    Vector3 v1 = tri.v1 - boxCenter;
    Vector3 v2 = tri.v2 - boxCenter;

    Vector3 e0 = v1 - v0;
    Vector3 e1 = v2 - v1;
    Vector3 e2 = v0 - v2;

    Vector3 triMin(
    min({v0.x, v1.x, v2.x}),
    min({v0.y, v1.y, v2.y}),
    min({v0.z, v1.z, v2.z})
    );

    Vector3 triMax(
    max({v0.x, v1.x, v2.x}),
    max({v0.y, v1.y, v2.y}),
    max({v0.z, v1.z, v2.z})
    );

    if (triMin.x > boxExtents.x || triMax.x < -boxExtents.x ||
       triMin.y > boxExtents.y || triMax.y < -boxExtents.y ||
       triMin.z > boxExtents.z || triMax.z < -boxExtents.z) {
        return false;
    }

    Vector3 normal = e0.cross(e1);
    float d = normal.dot(v0);
    float r = boxExtents.x * abs(normal.x) + boxExtents.y * abs(
       normal.y) + boxExtents.z * abs(normal.z);

    if (abs(normal.dot(v0)) > r) return false;

    float p0, p1, p2, rad;

    #define AXISTEST(axis_v, vA, vB, vC, proj_extents) \
    p0 = (vA).dot(axis_v); \
    p1 = (vB).dot(axis_v); \
    p2 = (vC).dot(axis_v); \
```

```
    rad = proj_extents; \
    if (max({p0, p1, p2}) < -rad || min({p0, p1, p2}) > rad) return false;

    Vector3 a00(0, -e0.z, e0.y);
    AXISTEST(a00, v0, v1, v2, boxExtents.y * abs(a00.y) + boxExtents.z * abs(a00.z));
    Vector3 a01(0, -e1.z, e1.y);
    AXISTEST(a01, v0, v1, v2, boxExtents.y * abs(a01.y) + boxExtents.z * abs(a01.z));
    Vector3 a02(0, -e2.z, e2.y);
    AXISTEST(a02, v0, v1, v2, boxExtents.y * abs(a02.y) + boxExtents.z * abs(a02.z));

    Vector3 a10(e0.z, 0, -e0.x);
    AXISTEST(a10, v0, v1, v2, boxExtents.x * abs(a10.x) + boxExtents.z * abs(a10.z));
    Vector3 a11(e1.z, 0, -e1.x);
    AXISTEST(a11, v0, v1, v2, boxExtents.x * abs(a11.x) + boxExtents.z * abs(a11.z));
    Vector3 a12(e2.z, 0, -e2.x);
    AXISTEST(a12, v0, v1, v2, boxExtents.x * abs(a12.x) + boxExtents.z * abs(a12.z));

    Vector3 a20(-e0.y, e0.x, 0);
    AXISTEST(a20, v0, v1, v2, boxExtents.x * abs(a20.x) + boxExtents.y * abs(a20.y));
    Vector3 a21(-e1.y, e1.x, 0);
    AXISTEST(a21, v0, v1, v2, boxExtents.x * abs(a21.x) + boxExtents.y * abs(a21.y));
    Vector3 a22(-e2.y, e2.x, 0); AXISTEST
    (a22, v0, v1, v2, boxExtents.x * abs(a22.x) + boxExtents.y * abs(a22.y));

    #undef AXISTEST

    return true;
}

void Octree::buildRecursive(const BoundingBox& box, const vector<Triangle>& triangles, int currentDepth) {
    if (currentDepth <= maxDepth) {
        lock_guard<mutex> lock(voxelMutex);
        nodesFormedCount[currentDepth]++;
    }

    vector<Triangle> intersectingTriangles;
    for (const auto& tri : triangles) {
        if (checkIntersection(box, tri)) {
```

```
            intersectingTriangles.push_back(tri);
        }
    }

    if (intersectingTriangles.empty()) {
        if (currentDepth <= maxDepth) {
            lock_guard<mutex> lock(voxelMutex);
            nodesSkippedCount[currentDepth]++;
        }
        return;
    }

    if (currentDepth == maxDepth) {
        lock_guard<mutex> lock(voxelMutex);
        voxels.push_back(box);
        return;
    }

    vector<BoundingBox> children = divideBox(box);
    for (int i = 0; i < 8; ++i) {
        buildRecursive(children[i], intersectingTriangles,
           currentDepth + 1);
    }
}

void Octree::voxelize(const BoundingBox& rootBox, const vector<
   Triangle>& triangles) {
    voxels.clear();

    nodesFormedCount.assign(maxDepth + 1, 0);
    nodesSkippedCount.assign(maxDepth + 1, 0);

    Vector3 size = rootBox.max - rootBox.min;
    float maxDim = max({size.x, size.y, size.z});
    Vector3 center = rootBox.getCenter();

    BoundingBox cubeRoot;
    cubeRoot.min = center - Vector3(maxDim/2, maxDim/2, maxDim
       /2);
    cubeRoot.max = center + Vector3(maxDim/2, maxDim/2, maxDim
       /2);

    vector<BoundingBox> children = divideBox(cubeRoot);
    vector<future<void>> futures;

    for (int i = 0; i < 8; ++i) {
        futures.push_back(async(launch::async, [this, childBox =
            children[i], &triangles]() {
            this->buildRecursive(childBox, triangles, 1);
```

```
        }));
    }

    for (auto& f : futures) {
        f.get();
    }
}
```

Listing 3: octree.cpp

3.4.2 geometry.hpp

File ini berisi struktur data yang digunakan dalam penyusunan voxel. Struktur data yang digunakan adalah `Vector3` yang merepresentasikan poligon segitiga untuk pembentukan model voxel serta struktur data `BoundingBox`.

```
#ifndef GEOMETRY_HPP
#define GEOMETRY_HPP

#include <cmath>
#include <algorithm>

struct Vector3 {
    float x, y, z;

    Vector3() : x(0), y(0), z(0) {}
    Vector3(float x, float y, float z) : x(x), y(y), z(z) {}

    // Operasi vektor dasar
    Vector3 operator+(const Vector3& v) const {
        return Vector3(x + v.x, y + v.y, z + v.z);
    }

    Vector3 operator-(const Vector3& v) const {
        return Vector3(x - v.x, y - v.y, z - v.z);
    }

    Vector3 operator*(float s) const {
        return Vector3(x * s, y * s, z * s);
    }

    Vector3 operator/(float s) const {
        return Vector3(x / s, y / s, z / s);
    }

    // Perkalian titik dan silang
    float dot(const Vector3& v) const {
        return x * v.x + y * v.y + z * v.z;
    }

```

```
    Vector3 cross(const Vector3& v) const {
        return Vector3(y * v.z - z * v.y, z * v.x - x * v.z, x * v.y
            - y * v.x);
    }

    // Panjang vektor dan normalisasi
    float length() const {
        return std::sqrt(x*x + y*y + z*z);
    }

    Vector3 normalize() const {
        float len = length();
        if (len > 0) return *this / len;
        return *this;
    }
};

struct Triangle {
    Vector3 v0, v1, v2;
};

struct BoundingBox {
    Vector3 min;
    Vector3 max;

    BoundingBox() {
        min = Vector3(1e9, 1e9, 1e9);
        max = Vector3(-1e9, -1e9, -1e9);
    }

    Vector3 getCenter() const {
        return Vector3((min.x + max.x) / 2.0f, (min.y + max.y) / 2.0f
            , (min.z + max.z) / 2.0f);
    }

    Vector3 getSize() const {
        return max - min;
    }
};
#endif
```

Listing 4: geomtery.hpp

3.4.3 main.cpp

File `main.cpp` adalah titik masuk program utama voxelization. Program ini akan mengubah sebuah file `.obj` menjadi versi `voxel`-nya dengan algoritma yang dijelaskan sebelumnya. Program `main.cpp` menampilkan banyak voxel yang terbentuk, banyak vertex yang terbentuk, banyak faces yang terbentuk, statistik octree, kedalaman octree, waktu

eksekusi, serta path file hasil voxelization disimpan.

```
#include <iostream>
#include <chrono>
#include <vector>
#include "obj_parser.hpp"
#include "octree.hpp"

using namespace std;

void convertVoxelsToTriangles(const vector<BoundingBox>& voxels,
    vector<Triangle>& out_triangles) {
    for (const auto& voxel : voxels) {
        Vector3 min = voxel.min;
        Vector3 max = voxel.max;

        Vector3 v0(min.x, min.y, min.z);
        Vector3 v1(max.x, min.y, min.z);
        Vector3 v2(max.x, max.y, min.z);
        Vector3 v3(min.x, max.y, min.z);
        Vector3 v4(min.x, min.y, max.z);
        Vector3 v5(max.x, min.y, max.z);
        Vector3 v6(max.x, max.y, max.z);
        Vector3 v7(min.x, max.y, max.z);

        out_triangles.push_back({v0, v3, v1}); out_triangles.
            push_back({v1, v3, v2});
        out_triangles.push_back({v5, v6, v4}); out_triangles.
            push_back({v4, v6, v7});
        out_triangles.push_back({v4, v7, v0}); out_triangles.
            push_back({v0, v7, v3});
        out_triangles.push_back({v1, v2, v5}); out_triangles.
            push_back({v5, v2, v6});
        out_triangles.push_back({v3, v7, v2}); out_triangles.
            push_back({v2, v7, v6});
        out_triangles.push_back({v4, v0, v5}); out_triangles.
            push_back({v5, v0, v1});
    }
}

int main(int argc, char* argv[]) {
    if (argc < 4) {
        cerr << "Penggunaan: " << argv[0] << " <input.obj> <
            kedalaman_octree> <output.obj>\n";
        return 1;
    }

    string inputFile = argv[1];
    int depth = stoi(argv[2]);
    string outputFile = argv[3];
```

```

    vector<Vector3> vertices;
    vector<Triangle> triangles;
    BoundingBox rootBox;

    cout << "Memuat file input...\n";
    if (!readObj(inputFile, vertices, triangles, rootBox)) {
        return 1;
    }

    auto start_time = chrono::high_resolution_clock::now();

    Octree octree(depth);
    octree.voxelize(rootBox, triangles);

    auto end_time = chrono::high_resolution_clock::now();
    chrono::duration<double, milli> elapsed = end_time - start_time;

    const auto& voxels = octree.getVoxels();

    vector<Triangle> voxelTriangles;
    convertVoxelsToTriangles(voxels, voxelTriangles);

    size_t totalVertices = voxels.size() * 8;
    size_t totalFaces = voxelTriangles.size();

    writeObj(outputFile, voxelTriangles);

    cout << "\nHASIL VOXELIZATION\n";
    cout << "Banyaknya voxel yang terbentuk: " << voxels.size() << "\n";
    cout << "Banyaknya vertex yang terbentuk: " << totalVertices << "\n";
    cout << "Banyaknya faces yang terbentuk: " << totalFaces << "\n";

    cout << "Statistik node octree yang terbentuk:\n";
    vector<int> nodesFormed = octree.getNodesFormed();
    for(int i = 1; i <= depth; i++) {
        cout << i << " : " << (i < nodesFormed.size() ? nodesFormed[i] : 0) << "\n";
    }

    cout << "Statistik node yang tidak perlu ditelusuri:\n";
    vector<int> nodesSkipped = octree.getNodesSkipped();
    for(int i = 1; i <= depth; i++) {
        cout << i << " : " << (i < nodesSkipped.size() ? nodesSkipped[i] : 0) << "\n";
    }

```

```
    cout << "Kedalaman octree: " << depth << "\n";
    cout << "Lama waktu program berjalan: " << elapsed.count() << "
        ms\n";
    cout << "Path dimana file .obj disimpan: " << outputFile << "\n";

    return 0;
}
```

Listing 5: main.cpp

3.4.4 viewer.cpp

Program ini merupakan titik masuk untuk membuka GUI viewer. Pada intinya di program ini juga terdapat `int main()` yang bertugas untuk menampilkan GUI viewer yang telah dibuat.

```
#include "raylib.h"
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <cmath>
using namespace std;

Model LoadObj(const string& filename) {
    Model model = { 0 };
    ifstream file(filename);
    if (!file.is_open()) return model;

    vector<Vector3> temp_vertices;
    vector<float> mesh_vertices;
    string line;

    while (getline(file, line)) {
        if (line.empty()) continue;
        istringstream iss(line);
        string prefix;
        iss >> prefix;

        if (prefix == "v") {
            Vector3 v;
            iss >> v.x >> v.y >> v.z;
            temp_vertices.push_back(v);
        }
        else if (prefix == "f") {
            string v1, v2, v3;
            iss >> v1 >> v2 >> v3;

            auto getIdx = [](const string& token) {
```

```
                size_t pos = token.find('/');
                string num = (pos == string::npos) ? token : token.substr(0, pos);
                return stoi(num) - 1;
            };

            int i1 = getIdx(v1), i2 = getIdx(v2), i3 = getIdx(v3);
            mesh_vertices.push_back(temp_vertices[i1].x); mesh_vertices.push_back(temp_vertices[i1].y); mesh_vertices.push_back(temp_vertices[i1].z);
            mesh_vertices.push_back(temp_vertices[i2].x); mesh_vertices.push_back(temp_vertices[i2].y); mesh_vertices.push_back(temp_vertices[i2].z);
            mesh_vertices.push_back(temp_vertices[i3].x); mesh_vertices.push_back(temp_vertices[i3].y); mesh_vertices.push_back(temp_vertices[i3].z);
        }
    }

    if (mesh_vertices.empty()) return model;

    float minX = 1e9, minY = 1e9, minZ = 1e9;
    float maxX = -1e9, maxY = -1e9, maxZ = -1e9;
    for (size_t i = 0; i < mesh_vertices.size(); i += 3) {
        float x = mesh_vertices[i], y = mesh_vertices[i+1], z = mesh_vertices[i+2];
        if (x < minX) minX = x; if (x > maxX) maxX = x;
        if (y < minY) minY = y; if (y > maxY) maxY = y;
        if (z < minZ) minZ = z; if (z > maxZ) maxZ = z;
    }

    float centerX = (minX + maxX) / 2.0f;
    float centerY = (minY + maxY) / 2.0f;
    float centerZ = (minZ + maxZ) / 2.0f;

    float sizeX = maxX - minX;
    float sizeY = maxY - minY;
    float sizeZ = maxZ - minZ;
    float maxExtent = sizeX;
    if (sizeY > maxExtent) maxExtent = sizeY;
    if (sizeZ > maxExtent) maxExtent = sizeZ;

    float targetSize = 25.0f;
    float scale = targetSize / (maxExtent == 0 ? 1.0f : maxExtent);

    for (size_t i = 0; i < mesh_vertices.size(); i += 3) {
        mesh_vertices[i]   = (mesh_vertices[i] - centerX) * scale;
        mesh_vertices[i+1] = (mesh_vertices[i+1] - centerY) * scale;
        mesh_vertices[i+2] = (mesh_vertices[i+2] - centerZ) * scale;
```

```
    }

    Mesh mesh = { 0 };
    mesh.vertexCount = mesh_vertices.size() / 3;
    mesh.triangleCount = mesh.vertexCount / 3;

    mesh.vertices = (float *)MemAlloc(mesh_vertices.size() * sizeof(
        float));
    mesh.normals = (float *)MemAlloc(mesh_vertices.size() * sizeof(
        float));
    mesh.colors = (unsigned char *)MemAlloc(mesh.vertexCount * 4 *
        sizeof(unsigned char));

    Vector3 lightDir = { 0.5f, 0.8f, 0.3f };
    float len = sqrt(lightDir.x*lightDir.x + lightDir.y*lightDir.y +
        lightDir.z*lightDir.z);
    lightDir.x /= len; lightDir.y /= len; lightDir.z /= len;

    for (size_t i = 0; i < mesh_vertices.size(); i++) {
        mesh.vertices[i] = mesh_vertices[i];
    }

    for (size_t i = 0; i < mesh.vertexCount; i += 3) {
        float v0x = mesh.vertices[i*3],     v0y = mesh.vertices[i
            *3+1],    v0z = mesh.vertices[i*3+2];
        float v1x = mesh.vertices[(i+1)*3], v1y = mesh.vertices[(i+1)
            *3+1], v1z = mesh.vertices[(i+1)*3+2];
        float v2x = mesh.vertices[(i+2)*3], v2y = mesh.vertices[(i+2)
            *3+1], v2z = mesh.vertices[(i+2)*3+2];

        float e1x = v1x - v0x, e1y = v1y - v0y, e1z = v1z - v0z;
        float e2x = v2x - v0x, e2y = v2y - v0y, e2z = v2z - v0z;

        float nx = e1y * e2z - e1z * e2y;
        float ny = e1z * e2x - e1x * e2z;
        float nz = e1x * e2y - e1y * e2x;

        float nlen = sqrt(nx*nx + ny*ny + nz*nz);
        if (nlen > 0.0f) { nx /= nlen; ny /= nlen; nz /= nlen; }

        float intensity = (nx * lightDir.x + ny * lightDir.y + nz *
            lightDir.z);
        intensity = 0.4f + (intensity + 1.0f) * 0.3f;
        if (intensity > 1.0f) intensity = 1.0f;

        unsigned char colorValue = (unsigned char)(220 * intensity);

        for (int j = 0; j < 3; j++) {
            mesh.normals[(i+j)*3] = nx;     mesh.normals[(i+j)*3+1] =
```

```
                ny;   mesh.normals[(i+j)*3+2] = nz;
            mesh.colors[(i+j)*4] = colorValue;
            mesh.colors[(i+j)*4+1] = colorValue;
            mesh.colors[(i+j)*4+2] = colorValue;
            mesh.colors[(i+j)*4+3] = 255;
        }
    }

    UploadMesh(&mesh, false);
    model = LoadModelFromMesh(mesh);
    return model;
}

int main(int argc, char* argv[]) {
    if (argc < 2) {
        cout << "Cara penggunaan: ./bin/viewer.exe <path_ke_file.obj
            >\n";
        return 1;
    }

    string filePath = argv[1];
    InitWindow(800, 600, "Obj Viewer");
    SetTargetFPS(60);

    Camera3D camera = { 0 };
    camera.position = (Vector3){ 15.0f, 15.0f, 15.0f };
    camera.target = (Vector3){ 0.0f, 0.0f, 0.0f };
    camera.up = (Vector3){ 0.0f, 1.0f, 0.0f };
    camera.fovy = 45.0f;
    camera.projection = CAMERA_PERSPECTIVE;

    Model model = LoadObj(filePath);
    bool isModelLoaded = (model.meshCount > 0);
    float camRadius = 30.0f;
    float camAngleX = 0.78f;
    float camAngleY = 0.78f;

    while (!WindowShouldClose()) {
        camRadius -= GetMouseWheelMove() * 10.0f;
        if (camRadius < 3.0f) camRadius = 3.0f;
        if (camRadius > 1000.0f) camRadius = 1000.0f;

        if (IsMouseButtonDown(MOUSE_BUTTON_LEFT)) {
            camAngleX -= GetMouseDelta().x * 0.01f;
            camAngleY += GetMouseDelta().y * 0.01f;
            if (camAngleY > 1.5f) camAngleY = 1.5f;
            if (camAngleY < -1.5f) camAngleY = -1.5f;
        }

```

```
        camera.position.x = camRadius * cos(camAngleY) * cos(camAngleX);
        camera.position.y = camRadius * sin(camAngleY);
        camera.position.z = camRadius * cos(camAngleY) * sin(camAngleX);
        camera.target = (Vector3){0.0f, 0.0f, 0.0f};

        BeginDrawing();
        ClearBackground(RAYWHITE);
        BeginMode3D(camera);
        DrawGrid(20, 1.0f);
        if (isModelLoaded) {
            DrawModel(model, (Vector3){ 0.0f, 0.0f, 0.0f }, 1.0f, WHITE);
        }
        EndMode3D();

        DrawText("Object Viewer", 10, 10, 20, DARKBLUE);
        DrawText(TextFormat("File: %s", filePath.c_str()), 10, 40, 10, DARKGRAY);

        if (!isModelLoaded) {
            DrawText("ERROR: FILE TIDAK DITEMUKAN ATAU KOSONG!", 10, 70, 20, RED);
            DrawText("Pastikan path file benar dan file sudah di-voxelize.", 10, 100, 15, GRAY);
        } else {
            DrawText("Status: SUKSES", 10, 70, 15, DARKGREEN);
        }

        DrawFPS(800 - 90, 10);
        EndDrawing();
    }

    if (isModelLoaded) UnloadModel(model);
    CloseWindow();
    return 0;
}
```

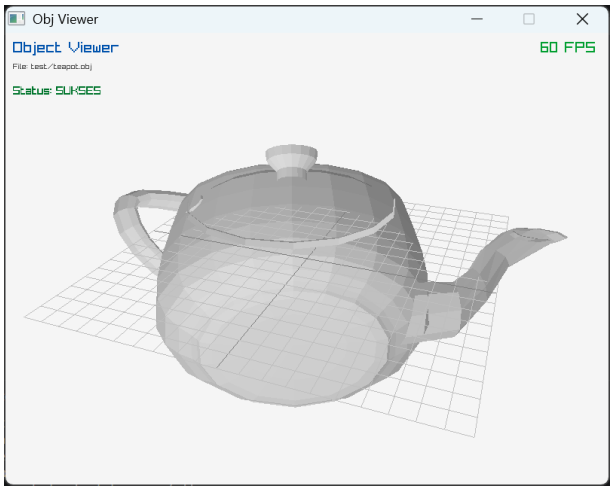
Listing 6: viewer.cpp

4 Eksperimen

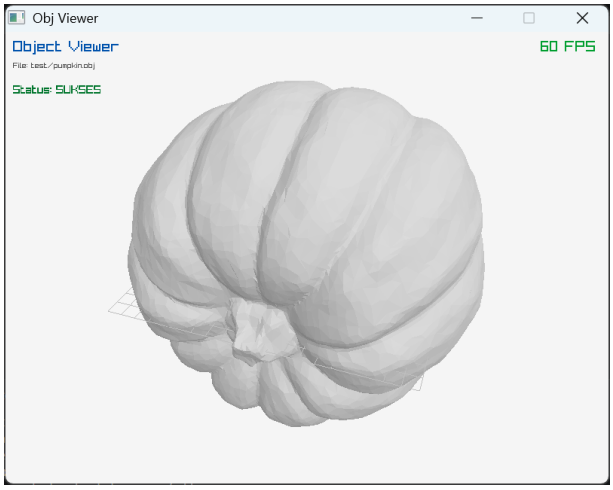
Bab ini memaparkan hasil pengujian fungsionalitas dan kinerja program yang telah dibuat. Eksperimen dilakukan dengan menguji program terhadap beberapa file tes uji yang telah disiapkan sebelumnya.

4.1 Testcases

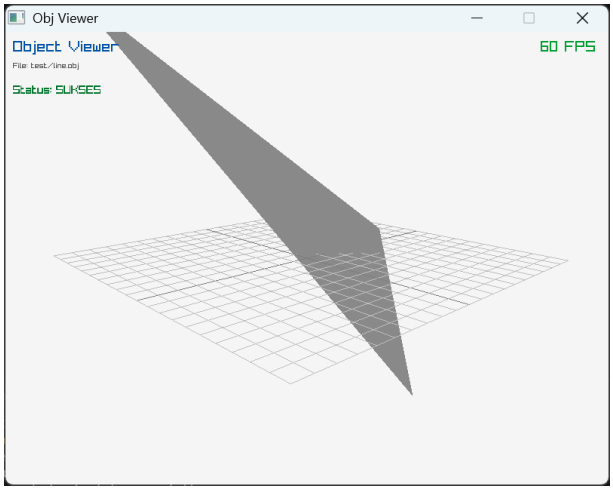
1. teapot.obj



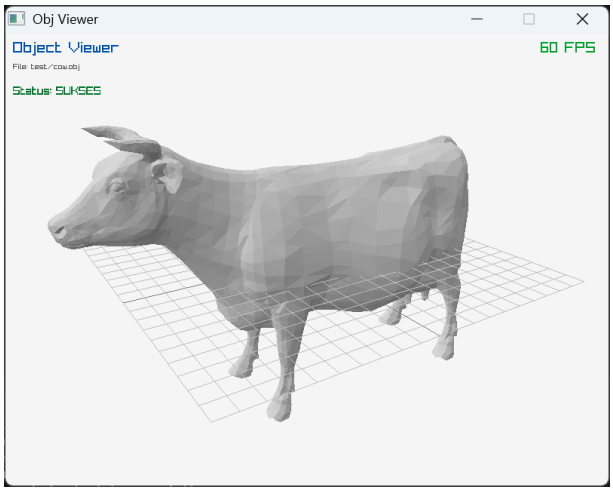
2. pumpkin.obj



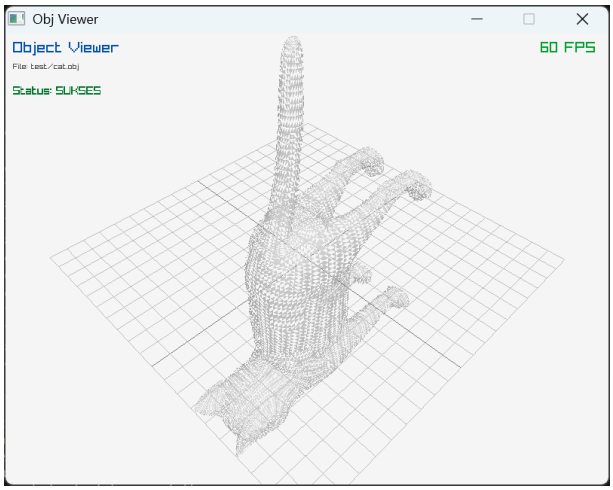
3. line.obj



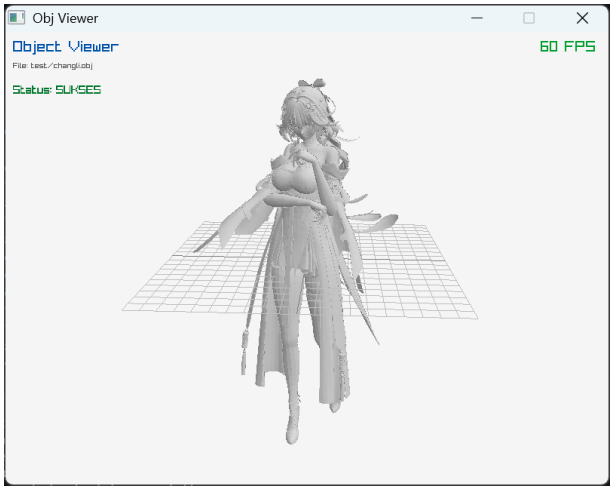
4. cow.obj



5. cat.obj



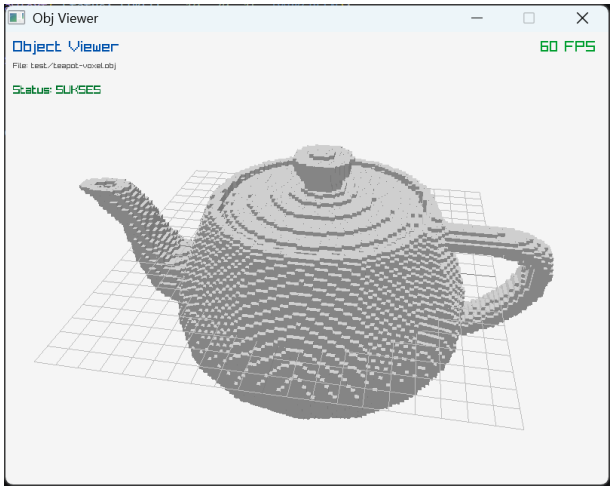
6. changli.obj



4.2 Hasil

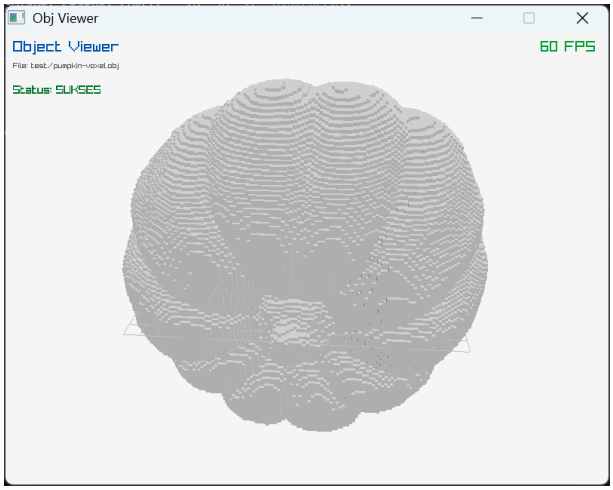
1. Hasil Voxelization teapot.obj

```
Memuat file input...

HASIL VOXELIZATION
Banyaknya voxel yang terbentuk: 29565
Banyaknya vertex yang terbentuk: 236520
Banyaknya faces yang terbentuk: 354780
Statistik node octree yang terbentuk:
1 : 8
2 : 64
3 : 192
4 : 864
5 : 3344
6 : 13984
7 : 58424
Statistik node yang tidak perlu ditelusuri:
1 : 0
2 : 40
3 : 84
4 : 446
5 : 1596
6 : 6681
7 : 28859
Kedalaman octree: 7
Lama waktu program berjalan: 33.964 ms
Path dimana file .obj disimpan: test/teapot-voxel.obj
```



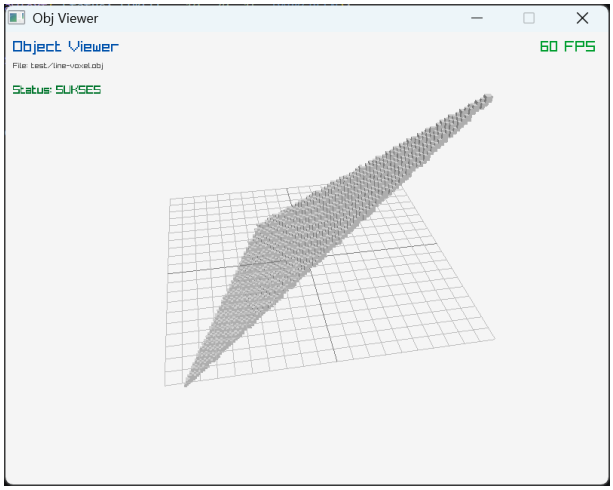
2. Hasil Voxelization pumpkin.obj

```
Memuat file input...

HASIL VOXELIZATION
Banyaknya voxel yang terbentuk: 74750
Banyaknya vertex yang terbentuk: 598000
Banyaknya faces yang terbentuk: 897000
Statistik node octree yang terbentuk:
1 : 8
2 : 64
3 : 448
4 : 2216
5 : 8984
6 : 37008
7 : 149096
Statistik node yang tidak perlu ditelusuri:
1 : 0
2 : 8
3 : 171
4 : 1093
5 : 4358
6 : 18371
7 : 74346
Kedalaman octree: 7
Lama waktu program berjalan: 77.359 ms
Path dimana file .obj disimpan: test/pumpkin-voxel.obj
```



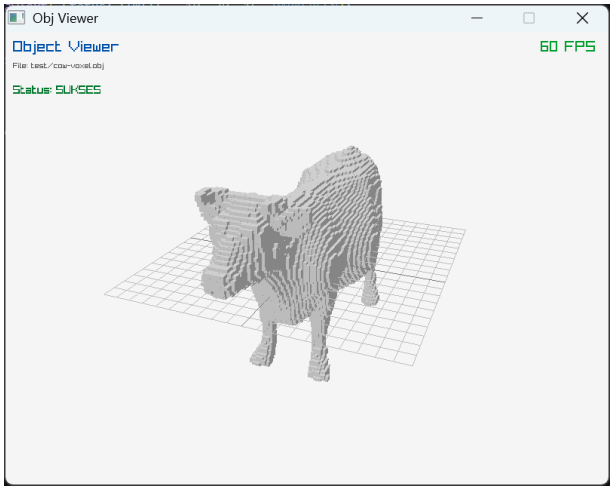
3. Hasil Voxelization line.obj

```
Memuat file input...

HASIL VOXELIZATION
Banyaknya voxel yang terbentuk: 3825
Banyaknya vertex yang terbentuk: 30600
Banyaknya faces yang terbentuk: 45900
Statistik node octree yang terbentuk:
1 : 8
2 : 56
3 : 112
4 : 256
5 : 736
6 : 2296
7 : 8144
Statistik node yang tidak perlu ditelusuri:
1 : 1
2 : 42
3 : 80
4 : 164
5 : 449
6 : 1278
7 : 4319
Kedalaman octree: 7
Lama waktu program berjalan: 3.162 ms
Path dimana file .obj disimpan: test/line-voxel.obj
```



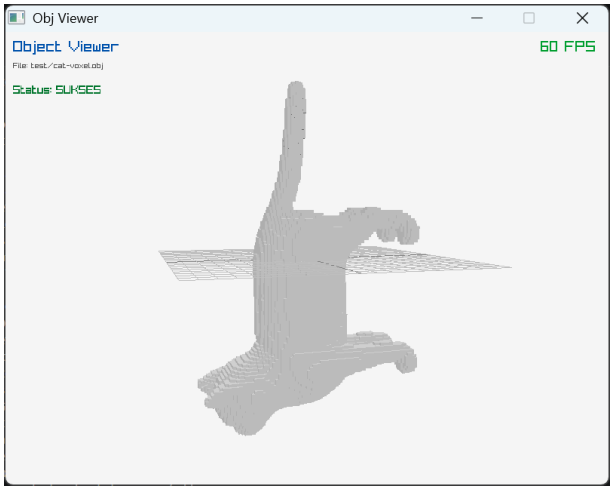
4. Hasil Voxelization cow.obj

```
Memuat file input...

HASIL VOXELIZATION
Banyaknya voxel yang terbentuk: 22864
Banyaknya vertex yang terbentuk: 182912
Banyaknya faces yang terbentuk: 274368
Statistik node octree yang terbentuk:
1 : 8
2 : 64
3 : 160
4 : 704
5 : 2656
6 : 10896
7 : 45016
Statistik node yang tidak perlu ditelusuri:
1 : 0
2 : 44
3 : 72
4 : 372
5 : 1294
6 : 5269
7 : 22152
Kedalaman octree: 7
Lama waktu program berjalan: 21.797 ms
Path dimana file .obj disimpan: test/cow-voxel.obj
```



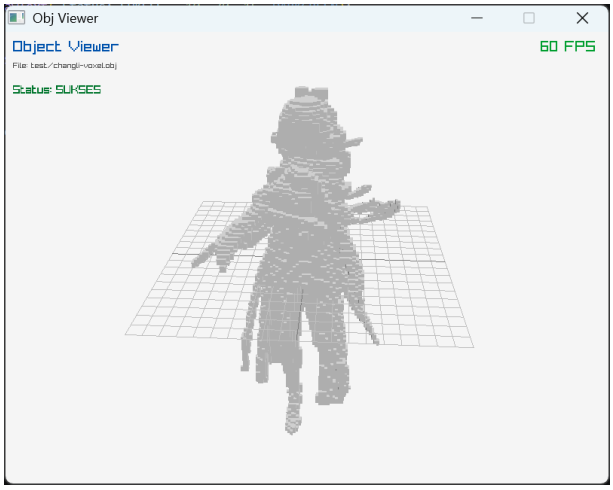
5. Hasil Voxelization cat.obj

```
Memuat file input...

HASIL VOXELIZATION
Banyaknya voxel yang terbentuk: 16031
Banyaknya vertex yang terbentuk: 128248
Banyaknya faces yang terbentuk: 192372
Statistik node octree yang terbentuk:
1 : 8
2 : 64
3 : 160
4 : 432
5 : 1968
6 : 7384
7 : 32256
Statistik node yang tidak perlu ditelusuri:
1 : 0
2 : 44
3 : 106
4 : 186
5 : 1045
6 : 3352
7 : 16225
Kedalaman octree: 7
Lama waktu program berjalan: 31.714 ms
Path dimana file .obj disimpan: test/cat-voxel.obj
```



6. Hasil Voxelization changli.obj

```
Memuat file input...

HASIL VOXELIZATION
Banyaknya voxel yang terbentuk: 25929
Banyaknya vertex yang terbentuk: 207432
Banyaknya faces yang terbentuk: 311148
Statistik node octree yang terbentuk:
1 : 8
2 : 56
3 : 152
4 : 544
5 : 2208
6 : 10080
7 : 45128
Statistik node yang tidak perlu ditelusuri:
1 : 1
2 : 37
3 : 84
4 : 268
5 : 948
6 : 4439
7 : 19199
Kedalaman octree: 7
Lama waktu program berjalan: 199.301 ms
Path dimana file .obj disimpan: test/changli-voxel.obj
```



5 Penutup

5.1 Kesimpulan

Berdasarkan hasil implementasi serta pengujian program, dapat ditarik kesimpulan sebagai berikut:

1. Algoritma *Divide and Conquer* merupakan algoritma yang digunakan dalam pemrosesesan grafik pada komputer. Algoritma ini mampu melakukan *voxelization* dengan struktur data *octree*.

2. Penerapan multithreading (menggunakan `std::async` dan pengamanan data melalui `std::mutex`) berhasil mengoptimalkan utilitas prosesor multi-core. Alur eksekusi independen pada pembelahan pohon tahap pertama membuat waktu eksekusi program sangat responsif, bahkan mencapai hitungan belasan milidetik pada model standar.

3. Uji coba pada objek *highpoly* tidak memberikan hasil yang efektif untuk hasil *voxelization*, hal ini dikarena objek *highpoly* memiliki detail yang kompleks.

5.2 Surat Cinta dari Waifu

Changli: Sudah kubilang jangan pakai C++!!!

Referensi

1. github.com/raysan5/raygui
2. www.raylib.com
3. informatika.stei.itb.ac.id/ rinaldi.munir/Stmik/2025-2026/07-Algoritma-Divide-and-Conquer-(2026)-Bagian1.pdf
4. stackoverflow.com/questions/8365013/reading-line-from-text-file-and-putting-the-strings-into-a-vector
5. stackoverflow.com/questions/9242415/passing-arguments-to-main
6. www.geeksforgeeks.org/dsa/octree-insertion-and-searching
7. wikipedia.org/wiki/Voxel
8. github.com/eisenwave/obj2voxel

Lampiran

- Tautan Repositori Github: github.com/An-Dafa/Tucil2-13524038

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	V	
2	Program berhasil di jalankan	V	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	V	
4	Program informasi-informasi atau report	V	
5	Program dapat menerima masukan .obj, dan mengembalikan output	V	
6	[Bonus] Program menggunakan *concurrency*	V	
7	[Bonus] Program dapat menampilkan visualisasi .obj	V	

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

An-Dafa Anza Avansyah